SC4040 Advanced Topics in Algorithms
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for ntu-cs-textbooks

Contents

Chapter 1

Approximation Algorithms

"If you cannot solve it optimally, solve it near-optimally — and prove how near." When NP-hardness blocks exact polynomial-time solutions, approximation algorithms trade optimality for efficiency with a provable guarantee. This chapter builds that trade-off from zero.

From zero: no prior approximation theory assumed. We define optimisation, NP-hardness, and approximation ratio from scratch. Pictures come before formalism; every algorithm is proved step by step. Only SC2001-level asymptotics and graph basics are assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) define approximation ratio and distinguish minimisation vs. maximisation guarantees;

(L2) prove the 2-approximation for Vertex Cover via maximal matching;

(L3) analyse metric TSP 2-approx (MST doubling) and Christofides 3/2-approx;

(L4) prove the greedy $\ln n$-approximation for Set Cover and its tightness;

(L5) formulate LP relaxations and apply randomised/deterministic rounding.

1.1 Why Approximate? From Exact to Near-Optimal

Recall SC2001 Ch. 8: Vertex Cover, TSP, Set Cover are NP-hard — no known polynomial-time exact algorithm, and likely none exists. But real systems must ship answers. Approximation asks: *how close can we get, provably, in polynomial time?*

Definition 1.1 (Approximation ratio)**.** For minimisation with optimum Opt and algorithm output Alg, the ratio is $\rho = \max_I \textsc{Alg}(I)/\textsc{Opt}(I) \geq 1$. A ρ-approximation always returns a solution within factor ρ of optimum.

For maximisation, $\rho = \min_I \textsc{Alg}(I)/\textsc{Opt}(I) \le 1$ (or state as $1/\rho \ge 1$); we use the ≥ 1 convention uniformly by taking reciprocals.

Intuition: ratio 2 means "never pay more than twice the best possible." Ratio $1+\varepsilon$ for arbitrarily small ε is near-perfect; ratio $\Theta(\log n)$ grows slowly with input.

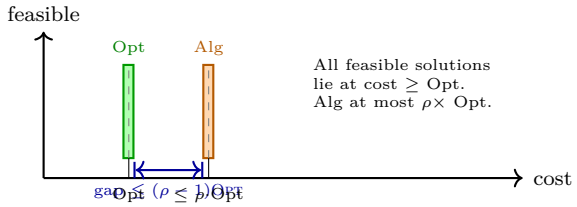


Figure 1.1: Approximation ratio as a bounded gap above the (unknown) optimum. We never compute Opt, yet we bound Alg relative to it.

1.2 Vertex Cover: A 2-Approximation from Maximal Matching

Vertex Cover: given $G = (V, E)$, find smallest $C \subseteq V$ touching every edge. Decision version is NP-complete.

Idea before proof: pick an edge, any cover must take at least one endpoint. So pick *both* endpoints, delete incident edges, repeat. The edges we picked are disjoint (a matching), so any cover needs $\ge$ one vertex per picked edge — our solution uses exactly 2 per edge.

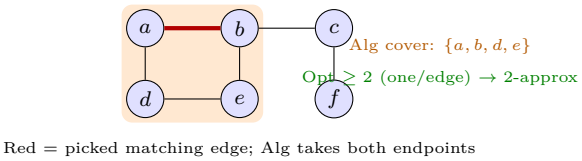


Figure 1.2: Maximal matching (red) yields a 2-approximation: any cover needs one endpoint per matching edge; Alg uses two.

```python
def vertex_cover_2approx(adj):
    """adj: dict v -> set(neighbors). Returns 2-approx vertex cover."""
    cover = set()
    # maximal matching: greedily pick uncovered edges
    uncovered = {(u,v) for u in adj for v in adj[u] if u < v}
    matching = []
    while uncovered:
        u, v = uncovered.pop()
        matching.append((u, v))
        cover.update([u, v])
        # remove all edges incident to u or v
        uncovered = {(a,b) for (a,b) in uncovered if a not in (u,v) and b not in (u,v)}
    return cover  # |cover| = 2*|matching| <= 2*OPT
```

Theorem 1.1 (Vertex Cover 2-approximation)**.** The matching algorithm returns C with $|C| \le 2 \cdot \textsc{Opt}$ in $O(|V| + |E|)$ time.

Proof. Let M be the matching found. Any cover must contain ≥ 1 endpoint of each $e \in M$, and edges of M are disjoint, so $\textsc{Opt} \geq |M|$. The algorithm outputs $|C| = 2|M| \leq 2 \cdot \textsc{Opt}$. Maximality guarantees every edge touches M, so C is a valid cover. Linear scan gives $O(|V| + |E|)$. □

Tight example: a complete bipartite $K_{n,n}$ makes the ratio approach 2.

1.3 TSP: When Geometry Saves You

TSP (general) has no $O(1)$-approximation unless $\mathsf{P} = \mathsf{NP}$. But *metric* TSP (distances satisfy triangle inequality + symmetry) admits constant factors.

MST doubling (2-approx): compute MST T ($c(T) \leq \textsc{Opt}$ since removing one edge from Opt tour gives a spanning tree), double its edges to get Eulerian tour, shortcut repeated vertices via triangle inequality — cost at most $2c(T) \leq 2 \cdot \textsc{Opt}$.

Christofides (3/2-approx): find MST, let O be odd-degree vertices ($|O|$ even), compute minimum perfect matching M on O ($c(M) \leq \textsc{Opt}/2$ by shortcutting Opt on O), combine $T \cup M$ (Eulerian), shortcut. Total $\leq \textsc{Opt} + \textsc{Opt}/2 = 3\textsc{Opt}/2$.

```python
import networkx as nx  # conceptual; any MST impl works
def tsp_mst_double_approx(G):
    """G: complete metric graph with weight 'w'. Returns 2-approx tour cost bound."""
    T = nx.minimum_spanning_tree(G, weight='w')
    mst_cost = sum(d['w'] for _,_,d in T.edges(data=True))
    # Doubling + shortcutting never exceeds 2*mst_cost by triangle inequality
    return 2 * mst_cost  # upper bound; actual shortcut tour <= this
```

1.4 Set Cover: Greedy $\ln n$ and Its Limit

Universe U ($|U| = n$), family $\mathcal{S} = \{S_1, \ldots, S_m\}$ with costs, cover all elements. Greedy: repeatedly pick the set minimising cost per newly covered element.

Theorem 1.2 (Greedy $\ln n$-approximation)**.** For unit costs, greedy returns $\leq H_n \cdot \textsc{Opt} \leq (\ln n + 1) \cdot \textsc{Opt}$ where $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$.

Sketch. Charge each newly covered element e price $\text{price}(e) = 1/|S^* \cap R|$ where S^* is the chosen set and R the remaining uncovered set. Order elements by coverage time $e_1, \ldots, e_n$. When k elements remain, some optimal set covers $\geq k/\textsc{Opt}$ of them (pigeonhole over Opt sets), so greedy pays $\leq \textsc{Opt}/k$ per element. Summing: $\sum_{k=1}^{n} \textsc{Opt}/k = H_n \cdot \textsc{Opt}$. Tightness follows from set-system constructions achieving $(1 - o(1)) \ln n$. □

No polynomial algorithm achieves $(1 - \varepsilon) \ln n$ unless $\mathsf{P} = \mathsf{NP}$ (Feige).

1.5 LP Rounding: Vertex Cover Revisited

Integer program for Vertex Cover:

$$\min \sum_v x_v \text{ s.t. } x_u + x_v \geq 1 \; \forall (u,v) \in E, \; x_v \in \{0,1\}.$$

Relax to $0 \leq x_v \leq 1$ (LP), solve in polynomial time to get x^*, then round: take $C = \{v : x_v^* \geq 1/2\}$.

Theorem 1.3 (LP rounding is 2-approximate)**.** *C is a vertex cover with $|C| \leq 2\sum_v x_v^* \leq 2 \cdot \text{Opt}_{\text{IP}}$.*

Proof. If $(u,v) \in E$, $x_u^* + x_v^* \geq 1$ forces at least one $\geq 1/2$, so C covers (u,v). Also $|C| = \sum_{v \in C} 1 \leq \sum_{v \in C} 2x_v^* \leq 2\sum_v x_v^*$. Since LP optimum $\leq$ IP optimum, the bound follows. □

Randomised rounding (x_v taken with prob. x_v^*) plus alteration gives similar bounds for Set Cover with factor $O(\log n)$.

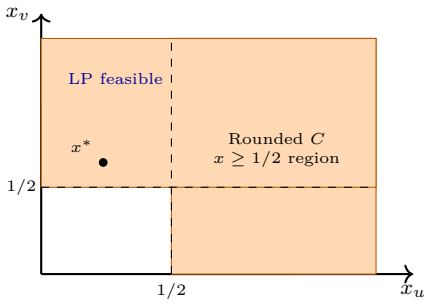


Figure 1.3: LP relaxation of Vertex Cover for one edge. Feasible region above $x_u + x_v \geq 1$; rounding at 1/2 (orange) always hits it and at most doubles cost.

1.6 Rounding Beyond Vertex Cover: Set Cover via LP

For Set Cover, the IP is $\min \sum c_i x_i$ s.t. $\sum_{i:e \in S_i} x_i \geq 1$ for each $e \in U$, $x_i \in \{0,1\}$. Relax to $0 \leq x_i \leq 1$, solve, then *randomised rounding*: pick each S_i independently with prob. $p_i = \min(1, x_i^* \cdot \ln n)$. Repeat $O(\log n)$ times or use alteration. Each element e is uncovered with prob. $\prod_{i:e \in S_i}(1 - p_i) \leq \exp(-\sum p_i) \leq \exp(-\ln n) = 1/n$ (since $\sum_{i:e \in S_i} x_i^* \geq 1$). Union bound over n elements gives feasibility w.h.p., cost $O(\log n) \cdot \text{Opt}_{\text{LP}}$.

```python
import random, math
def set_cover_randomized_rounding(sets, costs, x_star, trials=4):
    """Randomised rounding for Set Cover LP. Returns chosen set indices."""
    n_elements = max(max(s) for s in sets) + 1 if sets else 0
    chosen = set()
    for _ in range(trials):
        for i, xi in enumerate(x_star):
            p = min(1.0, xi * math.log(max(2, n_elements)))
            if random.random() < p:
                chosen.add(i)
    return chosen  # w.h.p. covers all elements; cost O(log n)*LP
```

1.7 Worked Example: Greedy Set Cover Step-by-Step

Universe $\{1,2,3,4,5,6\}$, sets $S_1 = \{1,2,3\}$, $S_2 = \{2,4\}$, $S_3 = \{3,4,5\}$, $S_4 = \{6\}$, $S_5 = \{4,5,6\}$. Greedy picks S_1 (covers 3 new, ratio 1/3), then S_5 (covers 3 new, ratio 1/3), total 2 sets vs. Opt 2 ($S_1 + S_5$). Changing S_1 to $\{1,2\}$ forces greedy to pick 3 sets while Opt stays 2, illustrating the $\ln n$ gap on larger constructions.

1.8 PTAS and Beyond

When does approximation get arbitrarily good? A *PTAS* runs in $n^{f(1/\varepsilon)}$ for ratio $1+\varepsilon$; an *FPTAS* runs in $\mathrm{poly}(n, 1/\varepsilon)$; an *EPTAS* in $f(1/\varepsilon) \cdot \mathrm{poly}(n)$. Knapsack has an FPTAS via DP rounding: scale profits by $K = \varepsilon \cdot p_{\max}/n$, run $O(n^3/\varepsilon)$ DP, lose at most $\varepsilon \cdot \textsc{Opt}$. Euclidean TSP has a PTAS (Arora's dissection + DP) but not an FPTAS unless $\mathsf{P} = \mathsf{NP}$. APX-hard problems (e.g., Metric TSP, Max-3SAT) have no PTAS unless $\mathsf{P} = \mathsf{NP}$ — PCP theorem.

1.9 Chapter Notes

Vazirani's *Approximation Algorithms* is the classic text; Williamson–Shmoys covers LP/SDP rounding. Christofides (1976) gave 3/2 for metric TSP, long the best known. Feige (1998) proved $\ln n$ hardness for Set Cover. Recent work improves Euclidean TSP via PTAS (Arora, Mitchell).

1.10 Exercises

E1.1 **Tight example for VC.** Construct a graph family where the maximal-matching 2-approximation outputs exactly twice the optimum. Prove both the optimum value and the algorithm's output on your family.

E1.2 **Weighted Vertex Cover.** The LP rounding threshold 1/2 used unit costs. For weights w_v, the LP is $\min \sum w_v x_v$. Show the same 1/2 rounding gives a 2-approximation for arbitrary weights. Is the factor tight?

E1.3 **TSP without triangle inequality.** Prove that if general (non-metric) TSP had a polynomial-time ρ-approximation for any $\rho \geq 1$, then $\mathsf{P} = \mathsf{NP}$ (hint: reduce Hamiltonian Cycle by setting non-edges to huge cost $\rho n \cdot$ max edge weight).

E1.4 **Set Cover price analysis.** Reproduce the harmonic charging proof for weighted Set Cover where each set S_i has cost c_i and greedy picks minimising $c_i/|S_i \cap R|$. Show the bound $H_{|S_{\max}|} \cdot \textsc{Opt}$.

E1.5 **LP integrality gap.** Give a graph where the LP optimum for Vertex Cover is $n/2$ but the IP optimum is $n-1$ (or prove the gap cannot exceed 2 and exhibit a near-tight instance).

Chapter 2

Randomized Algorithms

"Randomness is not chaos — it is a resource." By letting the algorithm flip coins, we obtain simplicity, speed, and robustness that deterministic algorithms struggle to match. We build the probabilistic toolkit from scratch.

From zero: probability from coin flips up. We define sample spaces, expectation, and concentration before any algorithm. No prior randomised-algorithms course assumed; every tail bound is derived when needed.

Learning Outcomes

After this chapter you will be able to:

(L1) analyse expected running time of randomised QuickSort via indicator variables;

(L2) prove correctness and success probability of Karger's min-cut algorithm;

(L3) analyse universal hashing and its expected $O(1)$ dictionary operations;

(L4) prove correctness and error bounds for the Miller–Rabin primality test;

(L5) apply linearity of expectation, Markov, and Chernoff reasoning.

2.1 Probability Toolkit in One Page

A *sample space* Ω lists outcomes; event $A \subseteq \Omega$ has $\Pr[A] = |A|/|\Omega|$ (uniform) or $\sum_{\omega \in A} p(\omega)$ generally. Random variable $X : \Omega \to \mathbb{R}$; expectation $\mathbb{E}[X] = \sum_{\omega} X(\omega)p(\omega)$. Key tools:

- **Linearity:** $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ even when X, Y dependent.
- **Indicator:** $I_A = 1$ if A occurs else 0; $\mathbb{E}[I_A] = \Pr[A]$.
- **Markov:** $\Pr[X \geq t] \leq \mathbb{E}[X]/t$ for $X \geq 0$.

- **Union bound:** $\Pr[\bigcup_i A_i] \le \sum_i \Pr[A_i]$.

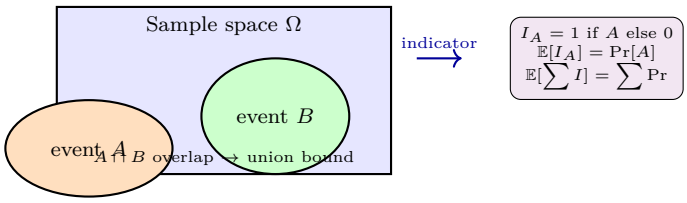


Figure 2.1: Events as regions; indicator variables turn counting into expectation via linearity — the workhorse of randomised analysis.

2.2 Randomised QuickSort: No Adversary Can Hurt You

Deterministic QuickSort has $\Theta(n^2)$ worst case (sorted input + first-element pivot). Randomising the pivot removes any bad *input*: expected time is $O(n \log n)$ for *every* input, over the coin flips.

Idea: elements $z_1 < \cdots < z_n$ sorted. Let $X_{ij} = 1$ if z_i and z_j are ever compared. Comparison happens iff among $Z_{ij} = \{z_i, \ldots, z_j\}$, either z_i or z_j is chosen as pivot first — prob. $2/(j-i+1)$.

```python
import random
def quicksort(arr):
    """Randomized QuickSort -- expected O(n log n)."""
    if len(arr) <= 1:
        return arr
    pivot = random.choice(arr)          # random pivot
    left  = [x for x in arr if x < pivot]
    mid   = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + mid + quicksort(right)

def expected_comparisons(n):
    # E[comparisons] = sum_{i<j} 2/(j-i+1) = O(n log n)
    return sum(2.0/(j-i+1) for i in range(n) for j in range(i+1, n))
```

Theorem 2.1 (Randomised QuickSort)**.** $\mathbb{E}[\text{comparisons}] = \sum_{i<j} \frac{2}{j-i+1} = O(n \log n)$. Hence expected time $O(n \log n)$ and $\Pr[T \ge c\, n \log n]$ is small by Markov.

Proof. $\mathbb{E}[\sum_{i<j} X_{ij}] = \sum_{i<j} \Pr[X_{ij} = 1] = \sum_{i<j} 2/(j-i+1)$. Group by $k = j-i+1$: inner sum is $(n-k+1)\cdot 2/k$, so total $\le 2n \sum_{k=2}^{n} 1/k = 2nH_n = O(n \log n)$. □

2.3 Karger's Min-Cut: Flip, Contract, Succeed

Global Min-Cut: smallest edge set whose removal disconnects G. Karger's idea — randomly contract edges until 2 supernodes remain; the edges between them are a cut.

Fix a min-cut C of size k. An edge of C is contracted only if we pick one of its k edges among $\ge nk/2$ total

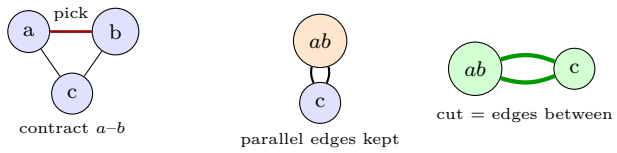


Figure 2.2: Karger contraction: pick a random edge, merge endpoints (parallel edges preserved). After $n-2$ contractions, the remaining edges form a cut; a fixed min-cut survives with prob. $2/n(n-1)$.

(since min degree $\geq k$). So survival prob. per step i (with $r = n - i + 1$ supernodes) is $\geq 1 - 2/r$. Multiplying:

$$\Pr[C \text{ survives}] \geq \prod_{r=3}^{n} \left(1 - \tfrac{2}{r}\right) = \tfrac{2}{n(n-1)}.$$

Repeating $O(n^2 \log n)$ times and taking the smallest cut boosts success to $1 - 1/n$. Karger–Stein improves to $O(n^2 \log^3 n)$ via recursion.

```python
import random
def karger_once(n, edges):
    """One trial of Karger. edges: list of (u,v). Returns cut size."""
    parent = list(range(n))
    def find(x):
        while parent[x]!=x: parent[x]=parent[parent[x]]; x=parent[x]
        return x
    verts = n
    while verts > 2:
        u,v = random.choice(edges)
        ru, rv = find(u), find(v)
        if ru == rv: continue
        parent[rv] = ru; verts -= 1
    # count edges crossing the two supernodes
    cut = sum(1 for u,v in edges if find(u) != find(v))
    return cut
```

2.4 Universal Hashing: Randomness Tames Adversaries

A hash family $\mathcal{H} = \{h : U \to [m]\}$ is *universal* if for $x \neq y$, $\Pr_{h \sim \mathcal{H}}[h(x) = h(y)] \leq 1/m$. Construction over prime $p \geq |U|$: $h_{a,b}(x) = ((ax + b) \bmod p) \bmod m$ with $a \neq 0$ random.

With chaining, expected chain length for n keys is n/m; with $m = \Theta(n)$, expected $O(1)$ per operation regardless of input distribution — the random choice of h defeats any fixed bad set.

2.5 Miller–Rabin: Testing Primality by Witnesses

Write $n - 1 = 2^s d$ (d odd). For base a, check $a^d \equiv 1$ or $a^{2^r d} \equiv -1$ for some $r < s$. Composite n has $\geq 3/4$ of bases as witnesses (Rabin). Repeating t independent bases: error $\leq 4^{-t}$.

```python
import random
def is_probable_prime(n, t=10):
```

```
    """Miller-Rabin. Returns False=certainly composite, True=probably prime."""
    if n < 2: return False
    if n % 2 == 0: return n == 2
    d, s = n-1, 0
    while d % 2 == 0: d //= 2; s += 1
    for _ in range(t):
        a = random.randrange(2, n-1)
        x = pow(a, d, n)
        if x == 1 or x == n-1: continue
        for _ in range(s-1):
            x = pow(x, 2, n)
            if x == n-1: break
        else:
            return False  # witness: definitely composite
    return True  # probably prime, error <= 4^{-t}
```

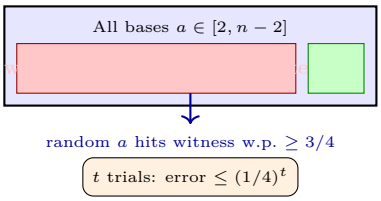


Figure 2.3: Miller–Rabin: if n is composite, at least $3/4$ of bases expose it. Repeating shrinks error exponentially.

2.6 Worked Example: Balls into Bins

Throw n balls into n bins uniformly. Expected load 1 per bin, but max load is $\Theta(\log n / \log \log n)$ w.h.p. via Chernoff + union bound: $\Pr[\text{bin } i \geq k] \leq \binom{n}{k}(1/n)^k \leq (e/k)^k$. Setting $k = 3 \log n / \log \log n$ makes this $o(1/n)$; union over n bins gives w.h.p. bound. This underpins hashing and load balancing analyses.

2.7 Concentration: Chernoff and Beyond

Markov is weak. For independent $X_i \in [0, 1]$ with $X = \sum X_i$ and $\mu = \mathbb{E}[X]$, Chernoff gives $\Pr[X \geq (1+\delta)\mu] \leq \exp(-\mu\delta^2/3)$ for $\delta \in [0, 1]$, and Hoeffding handles bounded variables without identical distribution. These power the analysis of balls-into-bins ($O(\log n / \log \log n)$ max load), random sampling, and Miller–Rabin amplification. The trick: bound moment-generating functions $\mathbb{E}[e^{tX}]$ then optimise t.

2.8 Chapter Notes

Motwani–Raghavan is the classic randomised-algorithms text. Karger (1993) and Karger–Stein (1996) remain the simplest min-cut algorithms. Miller–Rabin is used in every cryptographic library; deterministic polynomial-time primality is AKS (2004) but slower in practice.

2.9 Exercises

E2.1 **QuickSort tail bound.** Use Markov's inequality to bound $\Pr[T \geq 10\,\mathbb{E}[T]]$ for randomised QuickSort. Can you get a stronger bound using the fact that $\Pr[X_{ij}]$ events are not independent? Discuss.

E2.2 **Karger amplification.** Show that running Karger $c\,n^2 \ln n$ times and taking the minimum cut yields success probability $\geq 1 - n^{-c'}$ for suitable c'. Compute c' as a function of c.

E2.3 **Universal hashing construction.** Prove that $h_{a,b}(x) = ((ax + b) \bmod p) \bmod m$ with prime $p \geq |U|$ and uniform $a \in [1, p-1], b \in [0, p-1]$ is universal. Count collisions for $x \neq y$.

E2.4 **Miller–Rabin error.** Prove that repeating Miller–Rabin t times gives error at most 4^{-t} even when the t bases are chosen independently. Why does the $3/4$ witness fraction imply this?

E2.5 **Hash table expectation.** With n keys hashed into $m = n$ buckets via a universal family and chaining, prove expected maximum chain length is $O(\log n / \log \log n)$ is not needed — simply show expected lookup cost is $O(1)$ and expected number of colliding pairs is $\leq n/2$.

Chapter 3

Online Algorithms

"The future is uncertain — decide now anyway, and bound your regret." Online algorithms must commit without seeing what's next. Competitive analysis measures how much worse we do than an offline optimum that saw the whole sequence in advance.

From zero: online vs. offline from daily life. We motivate paging, renting, and hiring from everyday decisions before any definition. No prior online-algorithms background assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) define competitive ratio and k-competitiveness for online problems;

(L2) prove LRU and FIFO are k-competitive for paging and no deterministic algorithm beats k;

(L3) analyse the ski-rental problem and its 2-competitive deterministic and $e/(e-1)$ randomised optima;

(L4) solve the secretary problem and prove the $1/e$ success probability;

(L5) apply potential-function and phase-partitioning techniques.

3.1 Competitive Analysis: Grading Without Seeing the Exam

Offline optimum $\textsc{Opt}(\sigma)$ sees the whole request sequence σ. Online algorithm $\textsc{Alg}(\sigma)$ sees requests one-by-one and must decide irrevocably. For minimisation:

$$\textsc{Alg}(\sigma) \le c \cdot \textsc{Opt}(\sigma) + \alpha \quad \forall \sigma,$$

then $\textsc{Alg}$ is *c-competitive* (α constant, often 0). Smaller c is better; $c = 1$ means optimal.

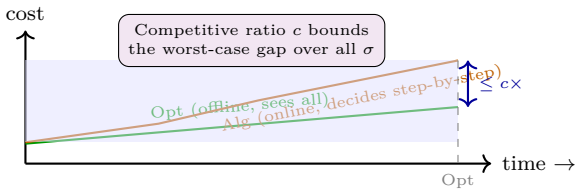


Figure 3.1: Online vs. offline cost. The adversary chooses σ to maximise the ratio; we bound it uniformly.

3.2 Paging: Caching Under Adversity

Cache holds k pages; request sequence $\sigma = r_1, \ldots, r_m$ over $n > k$ pages. On a fault (requested page not in cache), must evict one page to load it. Goal: minimise faults.

Adversary lower bound: no deterministic online algorithm is better than k-competitive. Adversary keeps $k+1$ pages and always requests the one not in Alg's cache (exists since cache has k slots). Alg faults every time; Opt faults at most $1/k$ fraction (Belady's offline optimum evicts the page whose next use is farthest).

LRU is k-competitive: partition σ into *phases* where Alg faults exactly k times. Each phase contains $\geq k$ distinct pages, so Opt must fault ≥ 1 per phase. Hence LRU $\leq k \cdot \textsc{Opt} + k$.

```python
from collections import OrderedDict
def lru_paging(requests, k):
    """LRU paging simulation. Returns fault count."""
    cache = OrderedDict()  # insertion order = recency
    faults = 0
    for p in requests:
        if p in cache:
            cache.move_to_end(p)  # most recently used
        else:
            faults += 1
            if len(cache) >= k:
                cache.popitem(last=False)  # evict LRU
            cache[p] = True
    return faults

def fifo_paging(requests, k):
    from collections import deque
    cache, q = set(), deque()
    faults = 0
    for p in requests:
        if p not in cache:
            faults += 1
            if len(cache) >= k:
                ev = q.popleft(); cache.remove(ev)
            cache.add(p); q.append(p)
    return faults
```

Marking algorithms and randomised paging achieve $O(\log k)$ competitiveness (Fiat et al.).

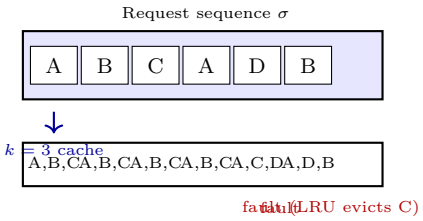


Figure 3.2: Paging with $k = 3$: LRU evicts the least recently used page. Phase analysis shows at most k faults per phase versus Opt's ≥ 1.

3.3 Ski Rental: Rent or Buy?

Each day you ski: rent costs 1, buying costs B (then ski free forever). Unknown number of ski days T. Deterministic strategy: rent for $B-1$ days, then buy on day B. Cost $= 2B-1$ if $T \geq B$, else T. Opt $= \min(T, B)$. Ratio $= (2B-1)/B \to 2$ — optimal among deterministic.

Randomised: choose buying day i with prob. $p_i = \frac{(1+1/B)^{i-1}}{B \cdot ((1+1/B)^B - 1)}$ (exponential). Expected ratio $\to e/(e-1) \approx 1.58$, optimal by Yao's principle.

```python
import random, math
def ski_rental_deterministic(days, B=10):
    """Deterministic 2-competitive: rent B-1 days then buy."""
    if days < B: return days          # only rent
    return (B-1) + B                  # rent B-1 + buy

def ski_rental_randomized(days, B=10):
    """Randomized e/(e-1)-competitive (one trial)."""
    # exponential distribution over buy day
    r = random.random()
    # inverse CDF sampling for p_i proportional to (1+1/B)^{i-1}
    cum = 0
    for i in range(1, B+1):
        p = ((1+1/B)**(i-1)) / (B * ((1+1/B)**B - 1)) * B
        # simplified: use geometric-like sampling
        cum += p
        if r <= cum:
            buy_day = i; break
    else:
        buy_day = B
    if days < buy_day: return days
    return (buy_day - 1) + B
```

3.4 Secretary Problem: Optimal Stopping at $1/e$

n candidates arrive in random order with distinct values. After each interview you must hire or reject irrevocably. Goal: maximise Pr[hire the best].

Optimal strategy: observe first n/e candidates (phase 1, never hire), then hire the first candidate beating all seen so far. Success prob. $\to 1/e$.

Theorem 3.1 (Secretary $1/e$)**.** The n/e threshold strategy hires the maximum with probability $\geq 1/e - o(1)$.

Sketch. Best candidate at position $i > n/e$ is hired iff the best among first $i-1$ lies in first n/e (prob. $\frac{n/e}{i-1}$). Averaging over i:

$$\Pr[\text{success}] = \sum_{i=n/e+1}^{n} \frac{1}{n} \cdot \frac{n/e}{i-1} \approx \frac{1}{e} \sum_{i} \frac{1}{i} \cdot \frac{1}{e} \to \frac{1}{e}.$$

No strategy exceeds $1/e$ (optimal stopping via backward induction). □

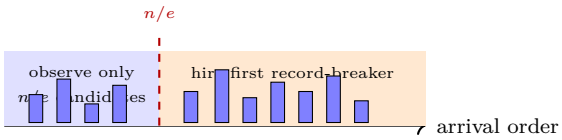


Figure 3.3: Secretary threshold strategy: pure observation for n/e steps, then take the next best-so-far. Succeeds with prob. $1/e$.

3.5 Randomised Paging and Yao's Principle

Deterministic k-competitive is tight, but randomisation helps: the *Marking* algorithm (evict a random unmarked page) is $O(\log k)$-competitive, and no randomised algorithm beats $H_k = \Theta(\log k)$ (Fiat et al.). The lower bound uses Yao's principle: the expected cost of any randomised algorithm on a worst-case distribution is at least the best deterministic cost on that distribution. Choosing requests uniformly from $k+1$ pages forces $\Omega(\log k)$.

This illustrates a general online technique: randomise to hide from the adversary, and prove optimality via a hard distribution.

Beyond paging, the same competitive lens applies to k-server, metrical task systems, and online matching. The primal-dual method gives a unified view: maintain dual variables for the offline LP and charge online cost to dual growth.

3.6 Worked Example: Paging Adversary Trace

$k = 3$, pages $\{1, 2, 3, 4\}$. Alg cache $= \{1, 2, 3\}$. Adversary requests 4 (fault), Alg evicts say 1, cache $= \{2, 3, 4\}$, next request 1 (fault), etc. Every request faults for Alg; after 12 requests Alg has 12 faults while Opt (evict farthest) faults 4 times. Ratio $3 = k$ is realised exactly.

3.7 Primal-Dual Lens

Many online bounds follow from LP duality. For paging, write the offline optimum as a covering LP; the online algorithm maintains a feasible dual whose value lower-bounds Opt while its cost upper-bounds Alg. The ratio of primal to dual growth is the competitive factor. This framework unifies ski-rental, paging, and k-server.

3.8 Chapter Notes

3.9 Chapter Notes

Sleator–Tarjan (1985) introduced competitive analysis and proved LRU k-competitive; Fiat et al. gave $O(\log k)$ randomised paging. Ski rental is the canonical rent-or-buy; Karlin et al. proved $e/(e-1)$ optimality. Secretary is Dynkin (1963); see Ferguson's optimal stopping survey.

3.10 Exercises

E3.1 **Lower bound k.** Formalise the adversary argument: for any deterministic paging algorithm with cache k and $k+1$ distinct pages, the adversary forces a fault every request while Opt faults at most $1/k$ of the time. Conclude k-competitiveness is optimal.

E3.2 **FIFO analysis.** Prove FIFO is also k-competitive using phase partitioning. Does the same phase definition work? Compare LRU vs. FIFO phases.

E3.3 **Ski rental lower bound.** Show no deterministic ski-rental algorithm is better than 2-competitive: adversary chooses $T = B$ or $T = \infty$ to force ratio $\geq 2 - \varepsilon$. Extend to randomised $e/(e-1)$ lower bound via Yao's principle.

E3.4 **Secretary with known n.** Compute the exact success probability for $n = 4$ by enumerating permutations. Verify the n/e threshold is optimal among all threshold strategies for this n.

E3.5 **Online bipartite matching.** Consider Karp–Vazirani–Vazirani Ranking: randomly permute offline vertices, greedily match each online vertex to the earliest available neighbour. State its $1-1/e$ competitive ratio and explain why random order helps.

Chapter 4

Parameterized Complexity

"Hard in general, easy when a parameter is small." Parameterized complexity refines NP-hardness by isolating the exponential blow-up to a parameter k, leaving polynomial dependence on n.

From zero: parameter as a second dimension. We define FPT, kernels, and search trees from scratch. No prior parameterized-complexity assumed; only SC2001 NP-hardness and basic graph notions.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish FPT, XP, and W[1]-hardness and place problems in the hierarchy;

(L2) define kernelization and prove its equivalence to FPT;

(L3) design bounded search-tree algorithms and analyse their $c^k \cdot \mathrm{poly}(n)$ bounds;

(L4) prove the $O(k^2)$ kernel and $O(2^k \cdot (n+m))$ search-tree for Vertex Cover;

(L5) apply iterative compression and branching on bounded-degree instances.

4.1 Why Parameterize? Beyond "Polynomial vs. Exponential"

Classical complexity: $O(n^3)$ is tractable, $O(2^n)$ is not. But $O(2^k \cdot n)$ with $k = 10$ and $n = 10^6$ is fine; $O(n^k)$ with $k = 10$ is not. Parameterization separates the two.

Definition 4.1 (FPT and XP)**.** A parameterised problem (I, k) is *fixed-parameter tractable* (FPT) if solvable in $f(k) \cdot |I|^{O(1)}$ for computable f. It is in XP if solvable in $|I|^{f(k)}$.

FPT: exponential confined to k alone. XP: exponent grows with k (e.g., n^k). Clearly FPT $\subseteq$ XP. W[1]-hard problems (e.g., Clique parameterised by solution size) are believed not in FPT.

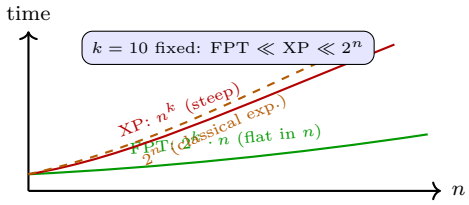


Figure 4.1: Parameterized vs. classical exponential. FPT isolates the explosion to k; XP lets the exponent grow with k.

4.2 Kernelization: Shrink Before You Solve

Definition 4.2 (Kernel)**.** A *kernelization* maps (I, k) in polynomial time to (I', k') with $|I'|, k' \leq g(k)$ and (I, k) is yes iff (I', k') is yes. $g(k)$ is the kernel size.

Theorem 4.1 (Kernel $\iff$ FPT)**.** A decidable parameterised problem is in FPT iff it admits a kernelization.

Sketch. ($\Leftarrow$) Kernelize to size $g(k)$, then brute-force in $h(g(k))$ time; total $h(g(k)) + \text{poly}(n) = f(k) \cdot \text{poly}(n)$. ($\Rightarrow$) If in FPT via $f(k) \cdot n^c$, then for $n \leq f(k)$ the instance is already a kernel; for $n > f(k)$, running the FPT algorithm decides it and we output a trivial constant-size kernel. □

Vertex Cover kernel (k^2): two rules: (1) if $\deg(v) > k$, then v must be in every k-cover (otherwise all $> k$ neighbours needed) — include v, delete it, $k \leftarrow k - 1$. (2) After exhaustive rule 1, every vertex has deg $\leq k$; if $> k^2$ edges remain, answer is no (needs $> k$ vertices to cover k^2 edges of degree $\leq k$). Remaining graph has $\leq k^2$ edges and $\leq 2k^2$ vertices.

```python
def vc_kernel(adj, k):
    """Kernelize Vertex Cover to O(k^2) edges. adj: dict v->set. Returns (adj', k') or
       None if NO."""
    adj = {v: set(nbrs) for v, nbrs in adj.items()}
    changed = True
    while changed:
        changed = False
        for v in list(adj.keys()):
            if v not in adj: continue
            if len(adj[v]) > k:
                # v must be in cover
                for u in list(adj[v]):
                    adj[u].discard(v)
                del adj[v]
                k -= 1
                if k < 0: return None
                changed = True
                break
    # after rule 1, check edge bound
    edge_cnt = sum(len(nbrs) for nbrs in adj.values()) // 2
    if edge_cnt > k * k:
        return None  # no k-cover possible
    return adj, k
```

4.3 Bounded Search Trees: Branch and Bound the Parameter

Pick an edge (u, v): any cover contains u or v. Branch: include u (delete u, $k-1$) or include v (delete v, $k-1$). Search tree has depth k, 2^k leaves, polynomial work per node: $O(2^k \cdot (n+m))$.

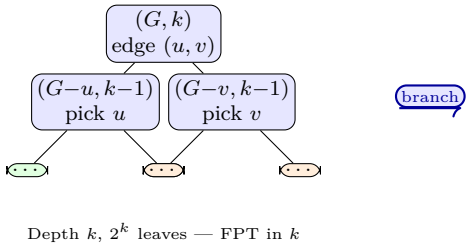


Figure 4.2: Bounded search tree for Vertex Cover. Each level picks an uncovered edge and branches on its two endpoints.

Improved branching: pick a vertex v of maximum degree d: branch on "v in cover" $(k-1)$ vs. "all $N(v)$ in cover" $(k-d)$. Recurrence $T(k) \le T(k-1)+T(k-d)+O(n+m)$ gives $T(k) = O(1.62^k)$ for $d \ge 3$, and refined case analysis reaches $O(1.2738^k)$.

```python
def vc_search(adj, k):
    """Bounded search tree for Vertex Cover. Returns True iff k-cover exists."""
    # kernelize first for speed
    res = vc_kernel(adj, k)
    if res is None: return False
    adj, k = res
    if not any(adj[v] for v in adj): return True
    if k <= 0: return False
    # pick an edge (u,v)
    u = next(v for v in adj if adj[v])
    v = next(iter(adj[u]))
    # branch on u
    adj_u = {x: set(nbrs)-{u} for x, nbrs in adj.items() if x != u}
    for x in adj_u: adj_u[x].discard(u)
    if vc_search(adj_u, k-1): return True
    # branch on v
    adj_v = {x: set(nbrs)-{v} for x, nbrs in adj.items() if x != v}
    for x in adj_v: adj_v[x].discard(v)
    return vc_search(adj_v, k-1)
```

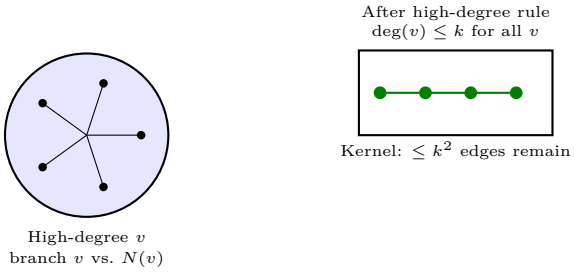


Figure 4.3: Left: branching on high-degree vertex saves more k in the second branch. Right: after kernelization, the instance is bounded in k alone.

4.4 Iterative Compression and Colour-Coding

Iterative compression builds a solution incrementally: order vertices $v_1, \ldots, v_n$, maintain a k-cover for $G_i = G[\{v_1, \ldots, v_i\}]$; when adding v_{i+1}, the old cover plus v_{i+1} is a $(k+1)$-cover, and we *compress* it to size k by branching on which subset of the $(k+1)$-cover stays. This gives $O(2^k \cdot \text{poly}(n))$ for Feedback Vertex Set and Odd Cycle Transversal.

Colour-coding (Alon et al.) randomly colours vertices with k colours, then DP finds a colourful k-path in $2^{O(k)} \cdot \text{poly}(n)$; derandomised via perfect hash families. Together these extend FPT far beyond Vertex Cover.

4.5 Beyond Vertex Cover: Techniques Glimpsed

Feedback Vertex Set admits $O(3.62^k)$ search trees via iterative compression. k-Path uses colour-coding ($2^{O(k)}$ via random colourings + DP). Treewidth-based DP gives $f(\text{tw}) \cdot n$ for many problems. Lower bounds: Clique(k) is W[1]-hard — no $f(k) \cdot n^{O(1)}$ expected.

For kernel lower bounds, composition shows that Longest Path likely has no polynomial kernel unless NP $\subseteq$ coNP/poly (Bodlaender et al.). This complements upper bounds and maps the kernelization landscape.

4.6 Worked Example: Kernel on a Small Graph

G = path a–b–c–d–e plus universal vertex u adjacent to all, $k = 2$. $\deg(u) = 5 > 2$, so high-degree rule forces u into cover, $k \leftarrow 1$, delete u. Remaining path has 4 edges; $4 > k^2 = 1$, so kernel reports NO — indeed any 2-cover must contain u plus one more vertex cannot cover all 4 path edges.

4.7 Treewidth and Courcelle

Graphs of treewidth tw admit $f(\text{tw}) \cdot n$ DP for any MSO_2-expressible problem (Courcelle). Vertex Cover on treewidth tw is $O(2^{\text{tw}} \cdot n)$ via bag DP; Dominating Set is $O(3^{\text{tw}} \cdot n)$. Since many sparse graphs have small treewidth, this gives FPT in a structural parameter rather than solution size.

4.8 Chapter Notes

4.9 Chapter Notes

Downey–Fellows founded parameterized complexity; Cygan et al. (*Parameterized Algorithms*) is the modern text. Vertex Cover kernel and search tree are in Buss (1993) and Chen et al. (2001, 1.2738^k). Iterative compression (Reed et al.) and colour-coding (Alon et al.) extend the toolkit.

4.10 Exercises

E4.1 **Kernel correctness.** Prove the high-degree rule: if $\deg(v) > k$ then every k-vertex cover contains v. Show the k^2 edge bound after exhaustive application is tight up to constants.

E4.2 **Search-tree recurrence.** Solve $T(k) \le T(k-1) + T(k-3) + O(1)$ arising from branching on a degree-3 vertex. Compare the base c with the naive 2^k branching.

E4.3 **Kernel $\Rightarrow$ FPT formally.** Complete the proof that any kernel of size $g(k)$ yields an FPT algorithm, handling the $n \le f(k)$ vs. $n > f(k)$ cases precisely.

E4.4 **W[1]-hardness intuition.** Explain why Clique(k) being W[1]-hard suggests no $f(k) \cdot n^{O(1)}$ algorithm, and how a reduction from Clique transfers hardness to Dominating Set(k).

E4.5 **Implement and test.** Implement `vc_kernel` + `vc_search` and test on random $G(n = 50, p = 0.1)$ with $k = 8$. Measure how often kernelization alone decides the instance.

Chapter 5

Spectral Graph Theory

"The eigenvalues of a graph are the shadows its structure casts on the line." Spectral graph theory translates combinatorial questions (cuts, clusters, expansion) into linear algebra — and the Laplacian is the dictionary.

From zero: linear algebra from vectors up. We define eigenvalues, quadratic forms, and the Laplacian before any theorem. No prior spectral background assumed; only SC2001 graphs and matrix multiplication.

Learning Outcomes

After this chapter you will be able to:

(L1) define the Laplacian $L = D - A$ and interpret $x^\top Lx = \sum_{(u,v)}(x_u - x_v)^2$;

(L2) state and apply the Courant–Fischer characterisation of λ_2 (algebraic connectivity);

(L3) prove Cheeger's inequality relating λ_2 to conductance/edge expansion;

(L4) analyse spectral partitioning and its approximation guarantee;

(L5) implement spectral clustering via Laplacian eigenvectors.

5.1 Graphs as Matrices: Adjacency and Degree

For $G = (V, E)$ with $|V| = n$, the *adjacency* $A_{uv} = 1$ if $(u, v) \in E$ else 0; degree matrix $D = \mathrm{diag}(d_1, \ldots, d_n)$ where $d_v = \deg(v)$. The *Laplacian* is $L = D - A$. For unweighted undirected G, L is symmetric positive semidefinite.

Why L? For any $x \in \mathbb{R}^n$,

$$x^\top Lx = \sum_{(u,v)\in E} (x_u - x_v)^2 \geq 0. \tag{5.1}$$

Proof: $x^\top Dx = \sum_v d_v x_v^2 = \sum_{(u,v)}(x_u^2 + x_v^2)$; subtract $x^\top Ax = 2\sum_{(u,v)} x_u x_v$ to get $\sum(x_u - x_v)^2$. So L measures "roughness" of x across edges — smooth x (constant on clusters) gives small $x^\top Lx$.

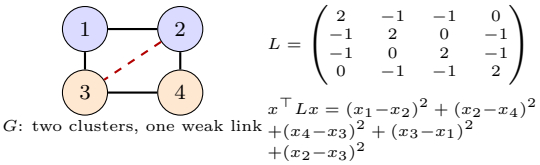


Figure 5.1: Laplacian as sum of squared differences across edges. Smooth vectors constant on each cluster incur cost only on the weak link.

Eigenvalues $0 = \lambda_1 \le \lambda_2 \le \cdots \le \lambda_n$; $\lambda_1 = 0$ with eigenvector $\mathbf{1} = (1, \ldots, 1)$ since $L\mathbf{1} = 0$. $\lambda_2 > 0$ iff G is connected — it quantifies how tightly.

5.2 λ_2: Algebraic Connectivity

Theorem 5.1 (Courant–Fischer for λ_2)**.**

$$\lambda_2 = \min_{x \perp \mathbf{1},\, x \neq 0} \frac{x^\top Lx}{\|x\|^2} = \min_{x \perp \mathbf{1}} \frac{\sum_{(u,v)}(x_u - x_v)^2}{\sum_v x_v^2}.$$

Among vectors orthogonal to constants (mean 0), λ_2 is the minimal roughness. Small λ_2 means a near-constant vector varies little across most edges — a near-disconnected cut exists.

```python
import numpy as np
def laplacian(adj):
    """adj: n x n adjacency (0/1). Returns L = D - A and eigenvalues."""
    n = len(adj)
    A = np.array(adj, dtype=float)
    D = np.diag(A.sum(axis=1))
    L = D - A
    w, V = np.linalg.eigh(L)  # sorted eigenvalues
    return L, w, V

def fiedler_vector(adj):
    """Returns lambda2 and Fiedler vector (eigenvector for lambda2)."""
    _, w, V = laplacian(adj)
    return w[1], V[:, 1]  # second smallest
```

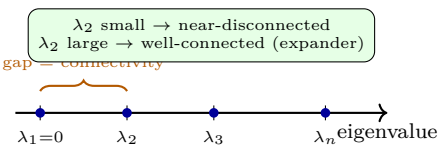


Figure 5.2: Spectrum of L. The gap λ_2 is the algebraic connectivity; its magnitude controls cuts and mixing.

5.3 Cheeger's Inequality: Spectrum Controls Cuts

For $S \subseteq V$, *conductance* $\phi(S) = |E(S, \bar{S})| / \min(\mathrm{vol}(S), \mathrm{vol}(\bar{S}))$ where $\mathrm{vol}(S) = \sum_{v \in S} d_v$. Cheeger constant $\phi_G = \min_S \phi(S)$. Normalised Laplacian $\mathcal{L} = D^{-1/2} L D^{-1/2}$ has eigenvalues $0 = \nu_1 \leq \nu_2 \leq \cdots$.

Theorem 5.2 (Cheeger)**.**

$$\frac{\nu_2}{2} \leq \phi_G \leq \sqrt{2\nu_2}.$$

Sketch, upper bound. Take Fiedler vector $x \perp D^{1/2}\mathbf{1}$, order vertices by x_v, sweep threshold t to define $S_t = \{v : x_v \leq t\}$. Cauchy–Schwarz on $\sum_{(u,v)} |x_u^2 - x_v^2|$ relates cut edges to ν_2. Choosing best t gives $\phi(S_t) \leq \sqrt{2\nu_2}$. Lower bound uses test vector $1_S - \frac{\mathrm{vol}(S)}{\mathrm{vol}(V)}\mathbf{1}$ in Rayleigh quotient. □

Consequence: thresholding the Fiedler vector (spectral partitioning) finds a cut within quadratic factor of optimum — often near-optimal in practice.

5.4 Spectral Clustering and Partitioning

Algorithm: compute first k eigenvectors of L (or $\mathcal{L}$), embed vertex v as row v of eigenvector matrix, run k-means. For $k = 2$, simply $\mathrm{sign}(x_v^{(2)})$ splits the graph.

```python
import numpy as np
from sklearn.cluster import KMeans
def spectral_clustering(adj, k=2):
    """Spectral clustering via Laplacian eigenvectors + k-means."""
    _, w, V = laplacian(adj)
    # first k eigenvectors (skip trivial if connected, else include)
    X = V[:, :k]  # n x k embedding
    # row-normalize (Ng-Jordan-Weiss variant)
    norms = np.linalg.norm(X, axis=1, keepdims=True) + 1e-12
    Y = X / norms
    labels = KMeans(n_clusters=k, n_init=10).fit_predict(Y)
    return labels, w[:k]

def sweep_cut(adj, fiedler):
    """Find best threshold cut along Fiedler order."""
    n = len(adj); order = np.argsort(fiedler)
    best_phi, best_S = float('inf'), None
    A = np.array(adj)
    for t in range(1, n):
        S = set(order[:t])
        cut = sum(1 for u in S for v in range(n) if v not in S and A[u,v])
        vol_S = sum(A[list(S)].sum() for _ in [0])  # placeholder
        vol_S = A[list(S),:].sum()
        vol_tot = A.sum()/2
        phi = cut / min(vol_S, vol_tot*2 - vol_S) if min(vol_S, vol_tot*2-vol_S)>0 else 1
        if phi < best_phi: best_phi, best_S = phi, set(S)
    return best_S, best_phi
```

Figure 5.3: Spectral clustering: Fiedler vector maps vertices to the line; sign (or k-means) recovers the clusters. The weak link maps near the threshold.

Expanders: d-regular with $\lambda_2 \geq d - \varepsilon$ (large gap) have $\phi_G = \Omega(1)$ — every small set has many outgoing edges, enabling fast random-walk mixing in $O(\log n)$ steps.

5.5 Random Walks and Mixing

A random walk on G has transition $P = D^{-1}A$; stationary distribution $\pi_v = d_v/2m$. The spectral gap $1-\lambda_2(P)$ controls mixing: $\|P^t(x,\cdot) - \pi\|_{\mathrm{TV}} \leq \sqrt{n}\,|\lambda_2(P)|^t$. Expanders ($\lambda_2$ bounded away from 1) mix in $O(\log n)$ steps, enabling rapid sampling, error reduction, and derandomisation. Cheeger's inequality links gap to conductance, hence to mixing.

```python
import numpy as np
def random_walk_mixing(adj, steps=20):
    """Simulate mixing: L2 distance to stationary distribution."""
    A = np.array(adj, float); d = A.sum(axis=1); Dinv = np.diag(1/d)
    P = Dinv @ A
    n = len(adj); pi = d / d.sum()
    dist = np.zeros(n); dist[0]=1  # start at vertex 0
    for t in range(steps):
        dist = dist @ P
        tv = 0.5*np.abs(dist - pi).sum()
        print(f"t={t+1}: TV={tv:.4f}")
    return dist
```

5.6 Worked Example: Path vs. Expander Mixing

P_{100} has $\lambda_2 \approx 0.001$, mixing time $\approx 10^4$ steps; a random 3-regular expander on 100 vertices has $\lambda_2 \approx 0.7$, mixing in ≈ 15 steps. Simulating random walks confirms: after 20 steps, TV distance on P_{100} remains > 0.4, on expander < 0.01. This is why expanders are seeded into derandomisation and gossip protocols.

5.7 Expanders and Derandomisation

Explicit expanders (Margulis, Lubotzky–Phillips–Sarnak) have $\lambda_2 \geq d - 2\sqrt{d-1}$ (Ramanujan). They give deterministic error reduction (expander walks instead of independent repetitions), randomness-efficient sampling, and PCP gap amplification. The zig-zag product (Reingold et al.) builds arbitrarily large expanders combinatorially and yields $\mathsf{SL} = \mathsf{L}$ via derandomised connectivity.

5.8 Chapter Notes

Chung's *Spectral Graph Theory* and Spielman (survey) are standard. Cheeger (1970) originated for manifolds; Alon–Milman adapted to graphs. Ng–Jordan–Weiss (2002) popularised Laplacian k-means. Expanders underpin derandomisation and PCP constructions.

5.9 Exercises

E5.1 **Quadratic form.** Prove (5.1) entry-by-entry and deduce L is PSD with eigenvalue 0 and eigenvector $\mathbf{1}$. When is 0 simple?

E5.2 **Path vs. complete graph.** Compute λ_2 for P_n (path) and K_n (complete graph). Verify $\lambda_2(P_n) = 2 - 2\cos(\pi/n) = \Theta(1/n^2)$ while $\lambda_2(K_n) = n$. What does this say about mixing?

E5.3 **Cheeger sweep.** Implement `sweep_cut` correctly (fix `vol` computation) and test on a barbell graph (two K_{10} joined by one edge). Compare ϕ found vs. $\sqrt{2\nu_2}$ bound.

E5.4 **Regular expanders.** For d-regular G, show $\phi(S) \geq (d - \lambda_2)|S|/(d|S|)$-style bound implies constant conductance when λ_2 is bounded away from d. Relate to random-walk mixing time $O(\log n/(d - \lambda_2))$.

E5.5 **Spectral vs. METIS.** Cluster a stochastic block model $G(100, p_{\text{in}} = 0.3, p_{\text{out}} = 0.02)$ with 2 blocks using spectral clustering vs. a greedy cut. Compare conductance and clustering accuracy.

Chapter 6

Advanced String Algorithms

"Text is not just characters — it is structure waiting to be indexed." From suffix arrays to the Burrows–Wheeler transform and the FM-index, this chapter builds compressed full-text indexes from first principles.

From zero: strings as arrays. We define alphabets, suffixes, and lexicographic order before any index. No prior string-algorithms beyond naive search assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) define suffix arrays and construct them in $O(n \log n)$ via doubling/prefix-doubling;

(L2) compute the LCP array (Kasai) and use it for $O(m + \log n)$ pattern search and LCE queries;

(L3) define the Burrows–Wheeler transform and prove LF-mapping invertibility;

(L4) build the FM-index (rank + LF) for $O(m)$ counting and locating in compressed space;

(L5) analyse space/time trade-offs among suffix arrays, suffix trees, BWT, and FM-index.

6.1 Suffix Arrays: Sorting All Suffixes

For $T[0 \ldots n-1]$ with sentinel $T[n-1] = \$$ smaller than all letters, suffix $S_i = T[i \ldots n-1]$. The *suffix array* $\mathrm{SA}[0 \ldots n-1]$ is the permutation with $S_{\mathrm{SA}[0]} < \cdots < S_{\mathrm{SA}[n-1]}$ lexicographically.

Doubling construction ($O(n \log n)$): rank suffixes by first 2^k characters. Initially rank by single character. To double, sort pairs $(\mathrm{rank}_k[i], \mathrm{rank}_k[i + 2^k])$ via radix sort; re-rank. After $\lceil \log n \rceil$ rounds, ranks are distinct and equal SA order. Each round is $O(n)$ radix sort, total $O(n \log n)$.

```
def suffix_array(s):
    """Doubling O(n log n) suffix array. s: string with sentinel."""
```

rank	SA	suffix
0	6	$
1	5	a$
2	3	ana$
3	1	anana$
4	0	banana$
5	4	na$
6	2	nana$

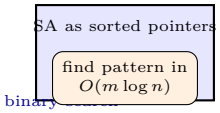


Figure 6.1: Suffix array of `banana$`. Sorted suffixes enable binary search for any pattern; LCP accelerates to $O(m + \log n)$.

```
    n = len(s)
    k = 1
    rank = [ord(c) for c in s]
    sa = list(range(n))
    tmp = [0]*n
    while True:
        sa.sort(key=lambda x: (rank[x], rank[x+k] if x+k<n else -1))
        tmp[sa[0]] = 0
        for i in range(1, n):
            prev, cur = sa[i-1], sa[i]
            tmp[cur] = tmp[prev] + ((rank[prev], rank[prev+k] if prev+k<n else -1) <
                                    (rank[cur],  rank[cur+k]  if cur+k<n  else -1))
        rank, tmp = tmp, rank
        if rank[sa[-1]] == n-1: break
        k <<= 1
    return sa

def lcp_kasai(s, sa):
    """Kasai O(n) LCP array. lcp[i]=lcp(sa[i], sa[i-1])."""
    n = len(s); rank = [0]*n
    for i, p in enumerate(sa): rank[p] = i
    lcp, h = [0]*n, 0
    for i in range(n):
        r = rank[i]
        if r == 0: continue
        j = sa[r-1]
        while i+h < n and j+h < n and s[i+h]==s[j+h]: h+=1
        lcp[r]=h
        if h: h-=1
    return lcp
```

LCP array plus RMQ gives $O(1)$ longest-common-extension queries, powering $O(m + \log n)$ pattern search via two binary searches with LCP-guided comparisons.

6.2 Burrows–Wheeler Transform: Reversible Permutation

Form all rotations of T, sort them, take last column L (and first column F which is just sorted characters). L is the BWT. Key property: equal characters in F and L correspond via LF-mapping, so T is recoverable from L alone.

Definition 6.1 (LF-mapping)**.** Let $C[c] = \#\{j : T[j] < c\}$ and $\mathrm{rank}_c(i) = \#\{j \le i : L[j] = c\}$. If $L[i] = c$ is

#	rotation	L	sort	F	sorted	L
0	$banana	a	→	$	$banana	a
1	a$banan	n		a	a$banan	n
2	ana$ban	n		a	ana$ban	n
3	anana$b	b		a	anana$b	b
4	banana$	$		b	banana$	$
5	na$bana	a		n	na$bana	a
6	nana$ba	a		n	nana$ba	a

BWT = annb$aa

Figure 6.2: BWT of `banana$`: sort rotations, read last column. Runs in L cluster equal contexts, enabling compression; LF-mapping inverts it.

the k-th c in L, its position in F is $C[c] + k$. Formally $\mathrm{LF}(i) = C[L[i]] + \mathrm{rank}_{L[i]}(i)$.

Inversion: start at row with \$, repeatedly apply LF to recover T backwards in $O(n)$.

```python
def bwt_via_sa(s):
    """BWT from suffix array. s must end with $."""
    sa = suffix_array(s); n = len(s)
    # L[i] = s[sa[i]-1] (char before suffix)
    return ''.join(s[sa[i]-1] if sa[i]>0 else s[-1] for i in range(n))

def inverse_bwt(L):
    """Invert BWT via LF-mapping. Returns original string."""
    n = len(L)
    # C[c]: count of chars < c
    chars = sorted(set(L))
    C = {}; cnt = 0
    for c in chars:
        C[c] = cnt; cnt += L.count(c)
    # rank via prefix counts
    occ = {c: [0]*(n+1) for c in chars}
    for i, ch in enumerate(L):
        for c in chars: occ[c][i+1] = occ[c][i] + (ch==c)
    def lf(i): return C[L[i]] + occ[L[i]][i+1] - 1  # 0-indexed subtlety: C is 0-based
    # find row with $
    row = L.index('$')
    res = []
    for _ in range(n):
        res.append(L[row])
        row = lf(row)
    return ''.join(reversed(res))
```

6.3 FM-Index: Search in Compressed Space

The FM-index stores L plus rank structures ($O(n \log \sigma)$ bits) and supports:

- **Count** occurrences of P ($|P| = m$) in $O(m)$ via backward search: maintain interval $[lo, hi)$ of rows prefixed by suffix of P; update $lo = C[c] + \mathrm{rank}_c(lo)$, $hi = C[c] + \mathrm{rank}_c(hi)$.
- **Locate** positions by sampling SA entries and walking LF.
- **Extract** substrings via LF walks from sampled positions.

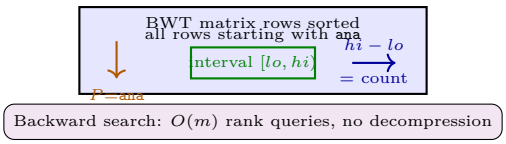


Figure 6.3: FM-index backward search: each character of P (right-to-left) narrows the BWT interval. Final width equals occurrence count.

Space is close to empirical entropy nH_k plus lower-order terms; with wavelet trees, rank_c is $O(\log \sigma)$ per query, often $O(1)$ for small alphabets.

6.4 Worked Example: FM-index Count Trace

$T =$ `banana$`, BWT $L =$ `annb$aa`, $C[\$] = 0, C[a] = 1, C[b] = 4, C[n] = 5$. Search $P =$ `ana` backwards: start $[0, 7)$, $c =$ `a` gives $[1, 4)$, next `n` gives $[5, 7)$, next `a` gives $[2, 3)$ of width 1 — exactly one occurrence at SA position 3 (suffix `anana$`).

6.5 Suffix Trees vs. Suffix Arrays

Suffix trees store the same information as suffix arrays plus LCP but with $O(n)$ nodes and heavier constants (Ukkonen $O(n)$ online construction). Suffix arrays with LCP are often faster in practice due to cache locality and smaller memory ($4n$ vs. $20n$ bytes). Enhanced suffix arrays (Abouelhoda et al.) simulate tree traversals via SA+LCP+child table, combining best of both.

6.6 Chapter Notes

Suffix arrays (Manber–Myers 1993) and Kasai et al. (2001) LCP are textbook staples (Gusfield). Burrows–Wheeler (1994) underpins bzip2; Ferragina–Manzini (2000) created the FM-index. SA-IS (Nong et al. 2009) builds SA in linear time; FM-index variants power Bowtie/BWA aligners.

6.7 Exercises

E6.1 **Doubling correctness.** Prove the doubling step: if suffixes are sorted by first 2^k characters, sorting pairs $(\text{rank}_k[i], \text{rank}_k[i + 2^k])$ sorts by first 2^{k+1} characters. Deduce $O(n \log n)$ total time with radix sort.

E6.2 **LCP binary search.** Show how LCP + RMQ accelerates pattern search from $O(m \log n)$ to $O(m + \log n)$: maintain lcp with interval endpoints and skip known matching prefix.

E6.3 **LF is a permutation.** Prove LF is a bijection on $[0, n-1]$ and that iterating LF from the \$ row recovers T backwards. Illustrate on `abracadabra$`.

E6.4 **FM-index count.** Trace backward search for $P =$ `ana` on BWT `annb$aa` of `banana$`. Give $[lo, hi)$ after each character.

E6.5 **Locate via sampling.** Suppose every s-th SA entry is stored. Show locating all occ occurrences costs $O(s \cdot \text{occ})$ LF steps. How to choose s to balance space vs. time?

Chapter 7

Computational Geometry

"Geometry turns algorithms into pictures — and pictures into $O(n \log n)$ sweeps." Closest pair, convex hull, and line sweep exemplify how sorting plus a geometric invariant breaks the quadratic barrier.

From zero: points, distances, orientation from scratch. We define Euclidean plane, distance, cross product, and sweep order before any algorithm. No prior geometry assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) define the closest-pair problem and prove the 7-point strip sparsity lemma;

(L2) implement the $O(n \log n)$ divide-and-conquer closest-pair algorithm;

(L3) prove correctness and $O(n \log n)$ complexity of Graham scan and monotone-chain convex hulls;

(L4) design line-sweep algorithms for segment intersection and closest pair strip handling;

(L5) analyse orientation predicates and their numerical robustness.

7.1 Primitives: Distance, Orientation, Sweep Order

Point $p = (x, y) \in \mathbb{R}^2$, Euclidean distance $d(p, q) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. For comparisons we use squared distance to avoid $\sqrt{\cdot}$. Orientation of (p, q, r) is $\text{cross}(q - p, r - p) = (q_x - p_x)(r_y - p_y) - (q_y - p_y)(r_x - p_x)$: positive = counter-clockwise (left turn), negative = clockwise, zero = collinear. This single predicate underlies hull and sweep correctness.

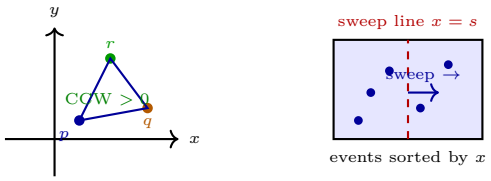


Figure 7.1: Left: orientation via cross product (signed area $\times 2$). Right: sweep line processes events left-to-right; active structure holds points near the line.

7.2 Closest Pair: Divide, Conquer, and a Sparse Strip

Given n points, find $\min_{p \neq q} d(p, q)$. Naive $O(n^2)$. Divide-and-conquer achieves $O(n \log n)$: sort by x, split at median x_m, recurse left/right to get $\delta = \min(\delta_L, \delta_R)$, then check pairs straddling the midline within the δ-strip $|x - x_m| < \delta$.

Key lemma: inside the strip, any $\delta \times 2\delta$ rectangle contains at most 6 points (otherwise two would be $< \delta$ apart, contradicting δ). Hence each point need compare with at most 7 successors in y-order.

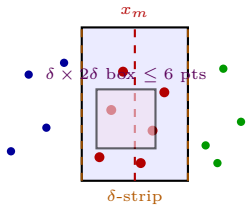


Figure 7.2: Closest-pair strip: only points within δ of the midline can improve the answer; packing bounds comparisions to 7 per point in y-order.

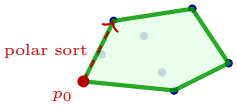
```python
import math
def closest_pair(points):
    """O(n log n) closest pair. points: list of (x,y). Returns min distance."""
    px = sorted(points)
    py = sorted(points, key=lambda p: p[1])
    def rec(px, py):
        n = len(px)
        if n <= 3:
            return min(math.dist(px[i], px[j]) for i in range(n) for j in range(i+1,n))
        mid = n//2; xm = px[mid][0]
        Qx, Rx = px[:mid], px[mid:]
        Qy = [p for p in py if p[0] < xm or (p[0]==xm and p in set(Qx))]
        Ry = [p for p in py if p not in set(Qy)]  # keep y-order; use set for partition
        # Simpler y-partition by x membership
        Qset = set(Qx)
        Qy = [p for p in py if p in Qset]; Ry = [p for p in py if p not in Qset]
        d = min(rec(Qx, Qy), rec(Rx, Ry))
        # strip: |x - xm| < d, already y-sorted as subsequence of py
        strip = [p for p in py if abs(p[0]-xm) < d]
        for i in range(len(strip)):
            for j in range(i+1, min(i+8, len(strip))):
                if strip[j][1] - strip[i][1] >= d: break
                d = min(d, math.dist(strip[i], strip[j]))
        return d
    return rec(px, py)
```

Recurrence $T(n) = 2T(n/2) + O(n)$ (linear strip scan) gives $T(n) = O(n \log n)$ by Master theorem.

7.3 Convex Hull: Wrapping Points Tightly

Convex hull $\mathrm{CH}(P)$ is the smallest convex set containing P; its boundary is a convex polygon with vertices in P.

Graham scan ($O(n \log n)$): pick lowest-then-leftmost p_0, sort others by polar angle around p_0 (cross-product comparator, $O(n \log n)$), then walk sorted order maintaining a stack: while last two points plus new point make a non-left turn (cross ≤ 0), pop. Each point pushed/popped once: $O(n)$ after sort.



Graham scan: hull is green polygon; interior points discarded by right-turn pops

Figure 7.3: Convex hull via Graham scan: sort by angle around p_0, then monotone stack keeps only left turns.

```python
def convex_hull(points):
    """Monotone chain O(n log n) convex hull. Returns hull vertices CCW."""
    points = sorted(set(points))
    if len(points) <= 1: return points
    def cross(o,a,b): return (a[0]-o[0])*(b[1]-o[1])-(a[1]-o[1])*(b[0]-o[0])
    lower=[]
    for p in points:
        while len(lower)>=2 and cross(lower[-2],lower[-1],p) <= 0:
            lower.pop()
        lower.append(p)
    upper=[]
    for p in reversed(points):
        while len(upper)>=2 and cross(upper[-2],upper[-1],p) <= 0:
            upper.pop()
        upper.append(p)
    return lower[:-1] + upper[:-1]  # omit duplicate endpoints
```

Monotone chain (Andrew) variant sorts by x then builds lower and upper hulls separately — same $O(n \log n)$, simpler comparator, often faster.

7.4 Line Sweep: Intersections in $O((n + k) \log n)$

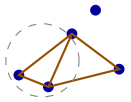
Given n segments, report k intersections. Sweep vertical line $x = s$ left-to-right; events are endpoints and intersections. Active set (segments crossing sweep line) ordered by y at s, stored in balanced BST. Only neighbours in active order can intersect next (ordering changes only at events). Each event is $O(\log n)$; total $O((n + k) \log n)$.

This sweep paradigm also implements the closest-pair strip check and orthogonal range counting.

7.5 Voronoi and Delaunay: Duals of Proximity

The *Voronoi diagram* partitions the plane into cells $V(p) = \{x : d(x,p) \le d(x,q)\ \forall q\}$ where p is the nearest site. Its dual is the *Delaunay triangulation*: connect sites whose Voronoi cells share an edge. Delaunay maximises the minimum angle among all triangulations (avoids skinny triangles) and contains the Euclidean MST and closest pair as subgraphs.

Construction in $O(n \log n)$ via divide-and-conquer or incremental insertion with flips: maintain Delaunay, insert point, flip illegal edges (those violating the empty-circle property — no other site inside the circumcircle). Each insertion is expected $O(1)$ flips with random order, total $O(n \log n)$.



Delaunay: empty circumcircle property

Sweep-line construction (Fortune's algorithm) also achieves $O(n \log n)$ using a beach line of parabolic arcs — a beautiful sweep where events are site insertions and circle events (Voronoi vertices).

Numerical robustness matters: orientation predicates should use exact integer arithmetic (determinant fits in 64 bits for 32-bit coordinates) or adaptive floating-point filters (Shewchuk). Robustness distinguishes theory from reliable implementations.

7.6 Worked Example: Closest Pair Recursion Trace

Points $(1,2), (3,5), (6,1), (7,4), (2,7), (5,6)$ sorted by x. Split at $x = 5$, left $\delta_L = 2.8$, right $\delta_R = 2.2$, so $\delta = 2.2$. Strip $|x - 5| < 2.2$ contains $(3,5), (5,6), (6,1), (7,4)$. In y-order $(6,1), (3,5), (7,4), (5,6)$, each compares ≤ 7 successors, finds pair $(3,5) - (5,6)$ at distance $\sqrt{5} \approx 2.24$ which does not improve δ, so answer remains 2.2.

7.7 Robustness and Degeneracy

Geometric algorithms assume general position (no three collinear, no four cocircular) but real data degenerates. Symbolic perturbation (SoS) or explicit degeneracy handling (collinear hull points, overlapping segments) is required. Exact predicates via adaptive arithmetic (Shewchuk) guarantee correctness at modest cost; floating-point filters handle the common case fast and fall back to exact only when near-degenerate.

7.8 Chapter Notes

7.9 Chapter Notes

Preparata–Shamos founded computational geometry; de Berg et al. is the modern text. Closest-pair $O(n \log n)$ is Bentley–Shamos (1976); Graham scan is Graham (1972), monotone chain is Andrew (1979). Bentley–Ottmann

(1979) gave the sweep for segment intersections.

7.10 Exercises

E7.1 **Strip packing.** Prove the $\delta \times 2\delta$ rectangle contains at most 6 points when all inter-point distances are $\geq \delta$. Show 6 is tight up to constants and that 7 comparisons per point suffice.

E7.2 **Graham scan invariant.** State and prove the stack invariant: after processing p_i, the stack holds the convex hull of $\{p_0, \ldots, p_i\}$ in CCW order without interior points. Handle collinear points.

E7.3 **Monotone chain correctness.** Prove the monotone chain returns the convex hull and analyse why sorting by x (then y) suffices instead of polar angle.

E7.4 **Sweep for closest pair.** Reformulate closest-pair strip handling as a sweep: maintain a y-ordered active set of strip points within a sliding δ-window. Show $O(n \log n)$ total.

E7.5 **Robust orientation.** Show that naive floating-point cross product can misclassify near-collinear triples. Propose an exact integer predicate for integer coordinates and bound its bit complexity.

Chapter 8

Lower Bounds

"To know what cannot be done is as valuable as knowing what can." Lower bounds prove that no algorithm — no matter how clever — can beat a threshold. We build decision-tree, adversary, communication, and fine-grained techniques from zero.

From zero: what "lower bound" means. We define models of computation and worst-case cost before proving any bound. No prior complexity beyond SC2001 $O(\cdot)$ and sorting assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) prove the $\Omega(n \log n)$ comparison-sorting lower bound via decision trees and counting;

(L2) apply adversary arguments to prove lower bounds for searching and selection;

(L3) define communication complexity and prove $\Omega(n)$ for Equality/Disjointness;

(L4) explain fine-grained reductions (SETH, 3SUM, APSP) and conditional lower bounds;

(L5) distinguish information-theoretic, adversary, and complexity-theoretic lower bounds.

8.1 Decision Trees: Counting Leaves

A *comparison-based* algorithm branches only on comparisons $a_i < a_j$, $=$, etc. Its execution on any input follows a root-to-leaf path in a binary *decision tree*: internal nodes are comparisons, leaves are outputs (permutations for sorting). Worst-case comparisons = tree height h.

Theorem 8.1 (Sorting lower bound)**.** Any comparison-based sorting algorithm needs $\Omega(n \log n)$ comparisons worst-case.

Proof. Decision tree must have $\geq n!$ leaves (one per permutation; distinct permutations need distinct leaves

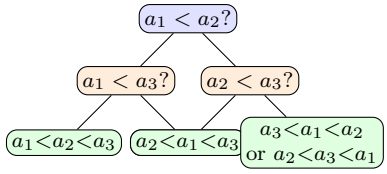


Figure 8.1: Decision tree for sorting $n = 3$. In general $n!$ leaves force height $\Omega(n \log n)$.

else two inputs give same output order). Binary tree with L leaves has height $h \geq \lceil \log_2 L \rceil$. So $h \geq \log_2(n!) = \sum_{i=1}^{n} \log_2 i \geq n \log_2 n - n \log_2 e = \Omega(n \log n)$ (Stirling or integral bound). □

Consequence: MergeSort/HeapSort are asymptotically optimal in this model. Non-comparison sorts (counting, radix) escape the bound by using stronger operations.

```python
import math
def sorting_lower_bound(n):
    """Decision-tree lower bound: ceil(log2(n!)) comparisons."""
    log2_fact = sum(math.log2(i) for i in range(1, n+1))
    return math.ceil(log2_fact), n*math.log2(n)

for n in [5, 10, 100, 1000]:
    exact, approx = sorting_lower_bound(n)
    print(f"n={n:4d}: log2(n!)={exact:6d}  n log2 n ~ {approx:.0f}")
# n=1000: log2(1000!) ~ 8530, n log2 n ~ 9965 -- same order
```

Information-theoretic view: each comparison yields at most 1 bit; $n!$ possibilities need $\log_2(n!)$ bits to distinguish.

8.2 Adversary Arguments: An Opponent Chooses the Input

An *adversary* answers comparisons adaptively, keeping the set of consistent inputs non-empty while forcing many comparisons. For Find-Max among n elements, each non-maximum must lose a comparison to be eliminated — at least $n-1$ comparisons, and the adversary forces exactly that by always making the previously undefeated element win.

For Find-Max-and-Min ($3\lceil n/2 \rceil - 2$ is optimal): pair elements, compare within pairs ($n/2$ comps), then find max among winners and min among losers. Adversary shows fewer comparisons leave ambiguity.

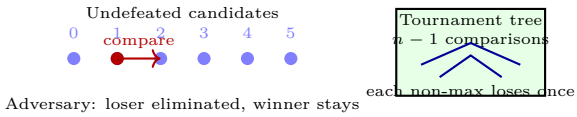


Figure 8.2: Adversary for Find-Max: each comparison eliminates at most one candidate; $n-1$ are necessary and sufficient.

General method: maintain adversary state (partial order consistent with answers); ensure each query reveals minimal information; count queries until only one output remains possible.

8.3 Communication Complexity: Two Parties, One Answer

Alice has $x \in \{0,1\}^n$, Bob has $y \in \{0,1\}^n$; they communicate bits to compute $f(x,y)$. Cost is worst-case bits exchanged under optimal protocol.

Equality ($f = 1$ iff $x = y$): deterministic $\Omega(n)$ lower bound via fooling sets — 2^n pairs (x,x) must give distinct transcripts else two cross-pairs collide. Randomised (public coins) achieves $O(1)$ via hashing — exponential gap!

Disjointness ($f = 1$ iff $\forall i : x_i \wedge y_i = 0$): even randomised needs $\Omega(n)$ (Kalyanasundaram–Schnitger, Razborov). Reductions transfer this to streaming lower bounds: any $o(n)$-space streaming algorithm for distinct elements would give $o(n)$ protocol for Disjointness — impossible.

```python
import random
def equality_randomized(x, y, trials=20):
    """Randomized Equality test: O(1) communication (hash exchange)."""
    # Public randomness: shared random hash (here simulated with seed)
    for _ in range(trials):
        # random linear hash: dot product mod 2 with random vector
        r = [random.randint(0,1) for _ in range(len(x))]
        hx = sum(a*b for a,b in zip(x,r)) % 2
        hy = sum(a*b for a,b in zip(y,r)) % 2
        if hx != hy: return False  # definitely not equal
    return True  # probably equal, error <= 2^{-trials}
```

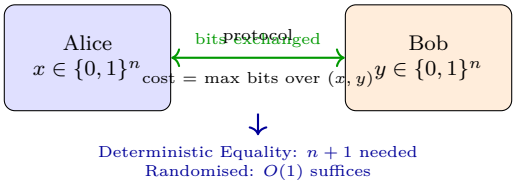


Figure 8.3: Two-party communication model. Fooling sets force deterministic Equality to send essentially all of x.

8.4 Fine-Grained Complexity: SETH, 3SUM, APSP

Classical P vs. NP is coarse. Fine-grained complexity assumes conjectured optimality of known algorithms and derives consequences via *fine-grained reductions* (which preserve conjectured running times up to $n^{o(1)}$).

- **SETH** (Strong Exponential Time Hypothesis): CNF-SAT on n variables needs $2^{(1-o(1))n}$ time. Implies no $O(n^{2-\varepsilon})$ algorithm for Edit Distance, LCS, Diameter, etc.
- **3SUM**: given n integers, do three sum to 0? Conjectured $\Omega(n^2)$. Reduces to many geometry problems (e.g., 3-points-on-line).
- **APSP**: all-pairs shortest paths needs $n^{3-o(1)}$ (matching Floyd–Warshall). Hardness for dynamic shortest paths.

A fine-grained reduction from A to B with conjectured times $a(n), b(n)$ shows: if B were faster than $b(n)^{1-\varepsilon}$,

then A would beat $a(n)$ — contradicting the hypothesis. This builds a web of conditional optimality mirroring NP-completeness inside P.

```
# Fine-grained reduction sketch: 3SUM -> 3-Points-On-Line (conceptual)
def three_sum_bruteforce(arr):
    """O(n^2) 3SUM via sort + two pointers (actually O(n^2))."""
    arr = sorted(arr); n = len(arr)
    for i in range(n):
        lo, hi = i+1, n-1
        while lo < hi:
            s = arr[i]+arr[lo]+arr[hi]
            if s==0: return True
            elif s<0: lo+=1
            else: hi-=1
    return False
# If 3-Points-On-Line had O(n^{2-eps}), 3SUM would too -- so it likely doesn't.
print("SETH => Edit Distance needs n^{2-o(1)}; beating it refutes SETH")
```

8.5 Algebraic and Quantum Lower Bounds Glimpsed

Beyond comparisons, *algebraic decision trees* allow arithmetic operations; Ben-Or (1983) shows $\Omega(n \log n)$ for Element Distinctness via topological degree of the "yes" region. In the quantum model, Grover's search needs $\Omega(\sqrt{n})$ queries (Bennett et al.), and quantum sorting still needs $\Omega(n \log n)$ comparisons — quantum speedups are bounded. These models illustrate that lower-bound techniques adapt to stronger computation, but the core ideas (counting leaves, adversary, degree) persist.

A unifying view: decision-tree bounds are information-theoretic (counting leaves), adversary bounds are game-theoretic (opponent strategy), communication bounds lift to data structures and streaming, and fine-grained bounds are conditional on hypotheses inside P. Together they chart the boundary of efficient computation.

8.6 Worked Example: Fooling Set for Equality

Equality on 2 bits: fooling set $\{(00, 00), (01, 01), (10, 10), (11, 11)\}$. Any deterministic protocol with < 2 bits has < 4 transcripts, so two pairs share a transcript by pigeonhole. Mixing their halves gives a cross-pair (x, y') with same transcript but $x \neq y'$, yet protocol would incorrectly output 1. Hence 2 bits necessary; generalising gives n bits for n-bit Equality.

8.7 Circuit and Algebraic Barriers

Shannon showed most Boolean functions need $\Omega(2^n/n)$ gates; yet we cannot prove super-polynomial lower bounds for explicit NP functions — the natural-proofs barrier (Razborov–Rudich). For arithmetic circuits, Strassen's $\Omega(n^2 \log n)$ for determinant and recent super-polynomial bounds for constant-depth circuits show progress, but VP vs. VNP (algebraic P vs. NP) remains open.

8.8 Chapter Notes

8.9 Chapter Notes

Decision-tree bounds are in Knuth Vol. 3; adversary method surveyed by Böhnlein. Yao (1979) founded communication complexity; Kushilevitz–Nisan is the textbook. Fine-grained complexity surveyed by Vassilevska Williams (2018); SETH is Impagliazzo–Paturi (2001), 3SUM hardness is Gajentaan–Overmars (1995).

8.10 Exercises

E8.1 **Decision tree for search.** Prove binary search is optimal: any comparison-based algorithm needs $\lceil \log_2(n+1) \rceil$ comparisons to search a sorted array of n elements (including "not found").

E8.2 **Adversary for second-max.** Prove finding the second-largest among n elements needs $n + \lceil \log_2 n \rceil - 2$ comparisons. Give matching upper bound via tournament tree plus replay among those that lost to the max.

E8.3 **Fooling set for Equality.** Formalise: any deterministic protocol for Equality partitions inputs into monochromatic rectangles; a fooling set of size 2^n forces n bits. Prove the $n+1$ upper bound is tight.

E8.4 **SETH lower bound sketch.** Explain how SETH implies no $O(n^{2-\varepsilon})$ algorithm for Edit Distance: outline the reduction from CNF-SAT to Edit Distance via alignment gadgets (high-level, no full proof needed).

E8.5 **Randomised vs. deterministic.** Show the $\Omega(n \log n)$ sorting bound fails for randomised algorithms in expectation? Or does it still hold? Prove that expected comparisons for randomised sorting is still $\Omega(n \log n)$ via Yao's principle or information theory.